Integrating with the Pesapal Gateway Service

This service is a shared proxy over CIO Africa's single Pesapal merchant account. Any internal application can use it to accept Pesapal payments - it has no idea what you're selling (tickets, merchandise, anything else) and doesn't store your orders. You own your own product/pricing logic and order records; this service only handles the Pesapal conversation.

Base URL

```
https://paymentapi.cioafrica.co
```

Authentication

Every request except `GET /health` requires an `x-api-key` header. You'll be issued a key by the CIO Africa engineering team, tied to your application's name for logging/revocation purposes. It is delivered to you out-of-band (not in this document) - treat it as a credential:

- Never commit it to source control.
- Never ship it inside a mobile app binary - it must be held by your own backend, which calls this service server-to-server.
- If it's ever exposed, ask for it to be rotated.

```
x-api-key: <your-assigned-key>
```

A missing or invalid key returns `401 { "error": "Unauthorized" }`.

The flow

1. Your backend computes the authoritative order (what's being bought, the price - never trust these values from a client app) and calls `POST /orders`.
2. You get back a `redirect_url`. Send the buyer there (a browser tab, or an in-app webview) to complete payment on Pesapal's hosted checkout page.
3. Pesapal redirects the buyer's browser to the `callback_url` you supplied
   - treat this as a "the buyer is probably done" hint, not proof of payment. Your `callback_url` page should show a pending/loading state and either poll `GET /orders/:id/status` or simply wait for step 4.
4. Independently, this service is notified by Pesapal (IPN) and forwards the payment result to the `webhook_url` you supplied in step 1. This is the reliable signal - it is triggered even if the

buyer closes the browser before the redirect happens. Your webhook handler is where you actually mark the order paid.

Steps 3 and 4 are two different notification paths for the same event. Build your own order-processing logic to be safe to run twice (see Idempotency below), a real `transaction/complete`-style redirect page AND a webhook can both be triggered for the same order.

What you need to build

`callback_url`, `cancellation_url`, and `webhook_url` in the `POST /orders` payload all point at **your own backend**, not at this gateway. This service never serves those routes for you - it only calls them. Concretely, your backend needs two publicly-reachable HTTPS endpoints:

1. **A webhook receiver** (`webhook_url`) - a normal server-to-server `POST` endpoint. No browser is involved; this gateway calls it directly when Pesapal notifies of a payment result. This is a JSON API route, e.g.:

```
// POST /api/pesapal-webhook
app.post('/api/pesapal-webhook', async (req, res) => {
  if (!isValidWebhookSecret(req.headers['x-webhook-secret'], WEBHOOK_SECRET)) {
    return res.status(401).json({ error: 'invalid signature' });
  }
  try {
    const order = await db.orders.findByReference(req.body.merchant_reference);
    if (order && order.status !== 'paid') {
      await db.orders.markPaid(order.id, req.body);
    }
  } catch (err) {
    console.error('webhook handling failed', err);
  }
  return res.status(200).json({ received: true }); // always 200, see below
});
```

2. **A callback page** (`callback_url`, and typically `cancellation_url` too) - a browser-facing route, because Pesapal redirects the buyer's *browser* there after checkout. If your checkout happens inside an in-app webview, this can be a minimal HTML page whose only job is to show a "confirming payment..." state and then close the webview / deep-link back into the app - it doesn't need to do the actual order-completion work itself (the webhook already does that). A bare-bones version:

```
// GET /checkout/complete?orderTrackingId=...
app.get('/checkout/complete', async (req, res) => {
  const status = await fetchGatewayStatus(req.query.orderTrackingId); // optional
poll
  res.send(renderPendingOrDoneHtml(status)); // simple HTML, or a deep link
});
```

Something has to be an internet-reachable server to receive #1 and host #2. It doesn't need to be elaborate - see `src/routes/api/payment/webhook/+server.js` and `src/routes/events/[slug]/transaction/complete/+page.server.js` in the `cio-africa-event-tickets-improved` app [sharing these files for reference] for a working reference of both, backed by our Directus backend.

Endpoints

`POST /orders`

Submit an order for payment.

```
{
  "order": {
    "id": "merch-order-4821",
    "description": "1x Conference T-Shirt (L), 1x Mug",
    "currency": "KES",
    "amount": 2500,
    "callback_url": "https://your-app.example/checkout/complete",
    "cancellation_url": "https://your-app.example/checkout/cancelled",
    "billing_address": {
      "first_name": "Jane",
      "last_name": "Doe",
      "email_address": "jane@example.com",
      "phone_number": "+254700000000"
    }
  },
  "webhook_url": "https://your-app.example/api/pesapal-webhook"
}
```

Field	Notes
`order.id`	Your own unique order identifier. Also used as `order_tracking_id` in the response for zero-amount orders.
`order.amount`	A number, in the given currency's minor-or-major unit as Pesapal expects (KES: whole shillings). `<= 0` short-circuits - no Pesapal call is made, useful for fully-discounted orders.
`order.callback_url` / `order.cancellation_url`	Where Pesapal redirects the buyer's browser after payment / cancellation. Must belong to you - this service doesn't use them.
`order.billing_address`	`first_name`, `last_name`, `email_address` are required; `phone_number` is recommended.
`webhook_url`	Where THIS service forwards the payment result once Pesapal notifies it (see below). Remembered for 24h - if payment happens outside that window, rely on polling `GET /orders/:id/status` instead.

Response (`200`):

```
{ "redirect_url": "https://pay.pesapal.com/...", "order_tracking_id": "..." }
```

`redirect_url` is `null` and no Pesapal call is made when `order.amount <= 0`.

Errors:

- `400` - invalid order payload or missing `webhook_url`, body is `{ "error": "<message>" }`.
- `502` - Pesapal itself failed, body is `{ "error": "<message>" }`.

`GET /orders/:orderTrackingId/status`

Returns Pesapal's transaction status verbatim (no reshaping). Use the `order_tracking_id` from the `POST /orders` response.

```
GET /orders/OTX123456789/status
x-api-key: <your-assigned-key>
```

Returns `502 { "error": "<message>" }` if the Pesapal call fails.

`POST /ipn`

Registered by this service with Pesapal directly. You never call this - it's how Pesapal notifies the gateway, which then forwards to your `webhook_url`.

Your webhook receiver

Your `webhook_url` receives a `POST` with:

```
{
  "order_tracking_id": "OTX123456789",
  "merchant_reference": "merch-order-4821",
  "payment_method": "Visa",
  "confirmation_code": "AB12CD34",
  "payment_status_description": "Completed",
  "payment_account": "4111********1111",
  "created_date": "2026-09-01T10:15:00Z",
  "amount": 2500,
  "currency": "KES"
}
```

Two identifiers are present because Pesapal's own tracking ID (`order_tracking_id`) is not the same value as the order ID you submitted (`merchant_reference` - this equals your `order.id` from `POST /orders`). **Look up your local order by `merchant_reference`**, since that's the one you control and generated yourself.

Your handler:

1. Verify the request came from this gateway (see below).
2. Look up your order by `merchant_reference`.
3. If you've already marked it paid, do nothing and return 200 (see Idempotency).
4. Otherwise, apply the payment result to your order and return 200.

Always return `200`, even on internal failure. This gateway does not retry - a non-200 or a hung request just means the notification is lost. Wrap your handler in a try/catch that still returns 200, and log failures for your own visibility instead of surfacing them to the caller.

Verifying the webhook is genuinely from this gateway

Every forwarded request carries a shared-secret header:

```
x-webhook-secret: <shared value, provided alongside your api key>
```

Compare it against the value you were given before trusting the payload. Use a constant-time comparison, not `===` / `!==`, to avoid timing attacks:

```
import { timingSafeEqual } from 'node:crypto';

function isValidWebhookSecret(receivedHeader, expectedSecret) {
    const a = Buffer.from(receivedHeader ?? '', 'utf8');
    const b = Buffer.from(expectedSecret, 'utf8');
    return a.length === b.length && timingSafeEqual(a, b);
}
```

This secret is shared across all consuming applications on this gateway - treat it with the same care as your API key.

Idempotency

Because both the browser redirect and the webhook can independently trigger your order-completion logic for the same order, and because Pesapal itself may send more than one IPN, **your handler must be safe to run twice.** Before applying a payment result, check whether the order is already marked paid/complete, and short-circuit if so - skip any side-effects (fulfillment triggers, notification emails, inventory decrements) that shouldn't happen more than once.

Zero-amount / fully-discounted orders

If `order.amount <= 0`, `POST /orders` returns immediately with `redirect_url: null` and does not contact Pesapal - there is nothing to pay, so no webhook will ever fire for that order either. Handle this case entirely in your own backend (mark it paid/free at order-creation time).

Limitations to design around

- **Single shared Pesapal merchant account.** All consuming apps share one set of Pesapal credentials - you cannot use a different merchant account through this service.
- **No persistent order storage here.** This service is a stateless proxy; it remembers your `webhook_url` for a given order for 24 hours (in memory) purely to route the IPN forward, then forgets it. You are the system of record for your own orders.
- **`webhook_url` is fetched server-side with no destination validation.** Only point it at your own infrastructure.